Lab 12 — ¿Por qué 8 núcleos no son 8× más rápido?

CC3069 Computación Paralela y Distribuida · Reporte · Partes 1 y 2 (integrante A)

Octubre 2026

Repositorio: https://github.com/DijanU/speedup

Archivo	Contenido
`investigacion.md`	Datos del equipo, marco teórico, modelo derivado y predicciones (escrito antes de medir)
`lab12.ipynb`	Compilación, ejecución de `amdahl` y `suma`, gráficas y cálculos (con salidas)
`resultados/`	CSV originales, `*_con_S_E.csv`, gráficas y repeticiones
`REPARTO.md`	Cómo se reparte el lab en la pareja

Reparto. El lab es en parejas. Este reporte cubre la preparación, la Parte 1 (Amdahl) y la Parte 2 (Ts honesto y granularidad). Las Partes 3 (desbalance) y 4 (Gustafson, segundo modelo) son del integrante B y se agregan en las secciones marcadas. La síntesis se completa entre los dos.

1. Datos del equipo

Dato	Valor
Equipo / CPU	MacBook Pro, Apple M5 Pro
Núcleos físicos / lógicos	15 / 15
Núcleos por tipo	5 “Super” (`perflevel0`, L2 de 16 MB) + 10 “Performance” (`perflevel1`, L2 de 8 MB)
Núcleos de eficiencia	0: en el M5 Pro los dos niveles son de rendimiento
Hyper-Threading / SMT	No (lógicos = físicos)
L1d / Memoria	64 KB por núcleo / 24 GB unificada
Compilador	Apple clang `-O2` + `libomp` (Homebrew)
p_max	15
Condiciones	Con batería (sin cargador), con otros procesos activos; ver la sección 5

Salida de `sysctl` (celda 3 del notebook, también en `evidencia_lab12.pdf`):

```
Apple M5 Pro
15
15
hw.perflevel0.name: Super          hw.perflevel0.physicalcpu: 5   hw.perflevel0.l2cachesize: 16777216
hw.perflevel1.name: Performance    hw.perflevel1.physicalcpu: 10  hw.perflevel1.l2cachesize: 8388608
hw.l1dcachesize: 65536             hw.memsize: 25769803776
```

2. Fórmulas usadas

S = Ts / Tp · E = S / p · C = p·Tp · F = S / C. Ts es `t_seq` (versión sin OpenMP) y Tp es `t_par`. Cada valor es la mediana de 5 repeticiones.

3. Parte 1 — La parte que no se puede repartir

3.1 Predicción vs. medición

fp	Amdahl S(15)	Mi predicción S(15)	Medido S(15)	Mejor S medido (p)	¿Acerté?
1.00	15.00	≈ 11	7.17	8.37 (p=9)	No: sobreestimé. Fue ideal hasta p=8 y luego se cayó
0.95	8.82	≈ 7.5	7.42	8.48 (p=13)	Sí en S(15), pero la curva es irregular
0.90	6.25	≈ 5.5	5.24	5.56 (p=12)	Cerca (−5 %)
0.75	3.33	≈ 3.1	2.84	3.06 (p=13)	Cerca en el mejor p; en p=15 empeora
0.50	1.88	≈ 1.8	1.67	1.76 (p=13)	Cerca

Esperaba que el quiebre de fp = 1.00 llegara en p = 5, al empezar a usar los núcleos “Performance”. No pasó: S = 6.06 con p = 6 y 8.10 con p = 8 (E ≈ 1.01). El quiebre llegó en p ≥ 9. No lo explica el tipo de núcleo, sino la competencia con otros procesos (ver 1.1).

3.2 Tabla de la Parte 1 (p_max = 15)

fp	S (p=2)	S (p=4)	S (p=15)	E (p=15)	p con F máxima	(p con Tp mínimo)
1.00	1.974	3.981	7.172	0.478	8	9
0.95	2.134	3.959	7.422	0.495	12	13
0.90	1.861	2.977	5.238	0.349	8	12
0.75	1.567	2.199	2.842	0.189	2	13
0.50	1.300	1.535	1.667	0.111	1	13

Nota sobre fp = 0.95: su `t_seq` (0.3425 s) salió ~12 % más alto que en las otras corridas (~0.30 s), así que S(1) = 1.13 y S(2) = 2.13 > 2. Es ruido de medición en Ts, no un speedup superlineal real.

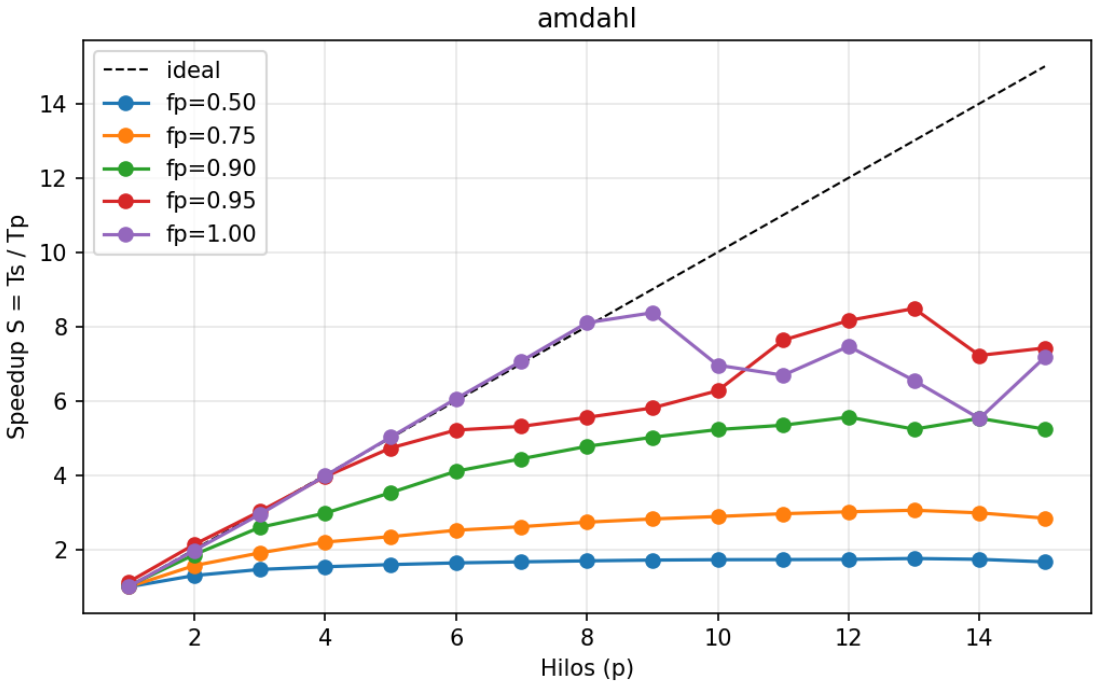


Figura 1: Speedup, modo amdahl

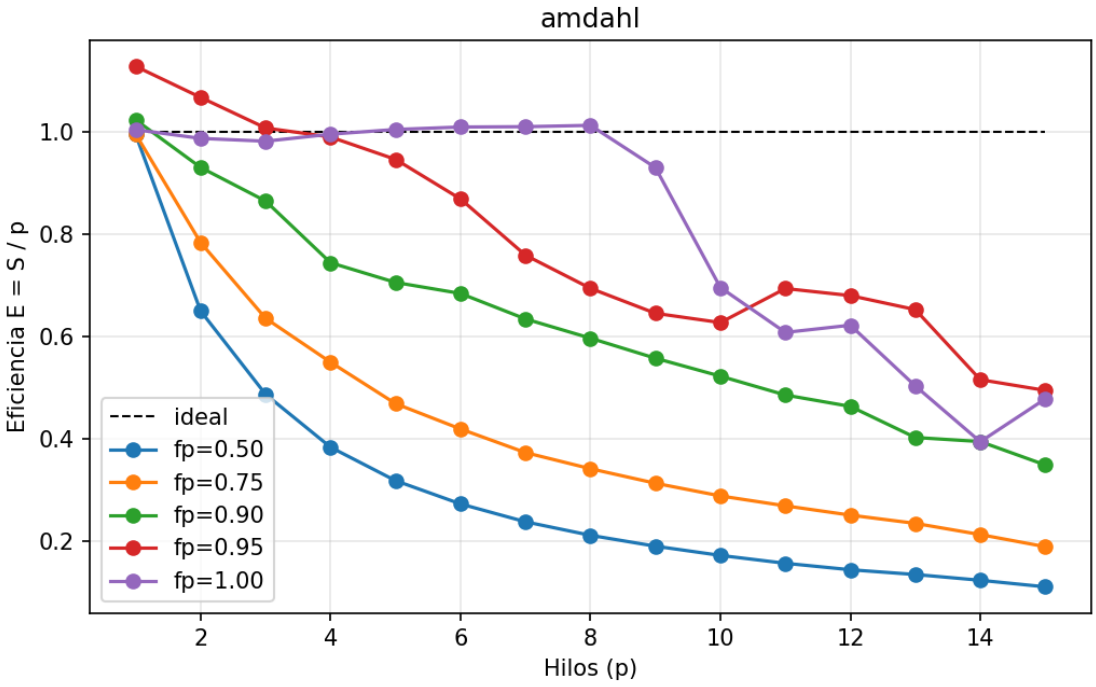


Figura 2: Eficiencia, modo amdahl

3.3 Cálculo a mano (dos filas, más una de control)

fp = 0.90, p = 4 — Ts = 0.311342 s, Tp = 0.104589 s

- S = 0.311342 / 0.104589 = 2.977
- E = 2.977 / 4 = 0.744
- C = 4 × 0.104589 = 0.418356 hilo·s
- F = 2.977 / 0.418356 = 7.116 s^{-1}

fp = 0.90, p = 15 — Ts = 0.311342 s, Tp = 0.059443 s

- S = 0.311342 / 0.059443 = 5.238
- E = 5.238 / 15 = 0.349
- C = 15 × 0.059443 = 0.891645 hilo·s
- F = 5.238 / 0.891645 = 5.874 s^{-1}

fp = 1.00, p = 15 — Ts = 0.302829 s, Tp = 0.042222 s

- S = 7.172 · E = 0.478 · C = 0.633330 hilo·s · F = 11.325 s^{-1}

Coinciden con `amdahl_con_S_E.csv` generado por `graficar.py`.

3.4 Preguntas

1.1 ¿Con fp = 1.00 se llegó a S = p? Hasta p = 8, sí: S = 1.97, 3.98, 6.06 y 8.10 con p = 2, 4, 6 y 8 (E entre 0.98 y 1.01). Con p = 15, no: S = 7.17 (E = 0.48), con una curva en serrucho entre p = 9 y 15. Tres corridas extra (`resultados/repeticiones/`) dieron S(14) = 12.6, 11.4 y 12.0 y S(15) = 9.7, 8.8 y 9.6. La caída es real pero varía mucho entre corridas. Causas:

1. Procesos en segundo plano y **`schedule(static)`**. Durante las corridas, `ps` mostró un proceso del sistema (`OtherUsersStorageExtension`) al 99 % de CPU, además de `WindowServer`, Terminal y el kernel de Jupyter. Con p = 15 no queda ningún núcleo libre. Si uno de los hilos comparte núcleo con ese proceso, avanza a la mitad de velocidad, y con `static` (bloques iguales) todos esperan al más lento en la barrera. Es la fuga η_b (desbalance), aunque provocada desde afuera. Por eso S(15) siempre es peor que S(14).
2. Núcleos heterogéneos. 5 “Super” + 10 “Performance”. En esta cadena de multiplicaciones la diferencia fue pequeña (no hubo quiebre en p = 6), pero con `static` cualquier diferencia de velocidad entre núcleos también es η_b.
3. Frecuencia y potencia. Con un solo hilo, el núcleo puede correr a su frecuencia máxima. Con 15 activos, y además con batería, el chip limita potencia y frecuencia. Ts se mide en la mejor condición y Tp en la peor. Esto no es ninguna de las cuatro fugas.
4. Costo de crear/despertar hilos. Son decenas de µs (ver la Parte 2) frente a 40 ms de trabajo: menos de 0.3 %. Sería η_c, pero aquí es despreciable.
5. Hyper-Threading: no aplica (no hay SMT).

1.2 Forma de la curva con fp = 0.75. Es cóncava y se aplana hacia la asíntota 1/(1 − 0.75) = 4. El hilo 2 aportó ΔS = +0.57. El hilo 8 aportó ΔS = +0.12, unas 5 veces menos. Desde p = 13 la ganancia es negativa (−0.07 y −0.14 en p = 14 y 15), porque el poco trabajo paralelo que queda ya no compensa el ruido y la sincronización.

1.3 Primer modelo (derivado). Tp = (1 − fp)·T1 + fp·T1/p ⇒

$$S(p) = \frac{T_1}{T_p} = \frac{1}{(1 - f_p) + f_p/p}$$

(Ley de Amdahl. La derivación completa está en `investigacion.md` §2.1.)

1.4 Predicción para fp = 0.90, p = 15. S = 1 / (0.10 + 0.90/15) = 1 / 0.16 = 6.25. Medido: 5.24, el 84 % del modelo. En p = 12 el modelo da 5.71 y se midió 5.56 (97 %). El modelo acierta bien hasta p ≈ 12, y en los últimos núcleos aparecen las pérdidas de 1.1.

1.5 Límite. Si p → ∞, S → 1/(1 − fp). Con fp = 0.90, S_max = 10. Con 1000 núcleos, S = 1/(0.1 + 0.0009) = 9.91.

1.6 fp despejado. fp = (1 − 1/S) / (1 − 1/p) = (1 − 1/5.238) / (1 − 1/15) = 0.8091 / 0.9333 = 0.867 < 0.90. Los 0.033 que faltan son tiempo que se comporta como si fuera secuencial, aunque el código no lo sea: los hilos esperando al más lento por procesos ajenos y núcleos distintos (η_b), más la creación de hilos y la barrera (η_c, pequeño). La

caída de frecuencia con todos los núcleos activos no es fuga. Para los otros fp: 0.922 (1.00), 0.927 (0.95), 0.694 (0.75) y 0.429 (0.50). Siempre está por debajo del valor nominal.

1.7 Efectividad con fp = 0.90. F es máxima en p = 8 ($F = 9.15\ s^{-1}$). El menor tiempo se da en p = 12 (Tp = 0.0560 s). No es el mismo p: de 8 a 12 hilos el tiempo baja solo 14 % (0.0652 → 0.0560 s), pero el costo sube 29 %. El modelo predecía $p^* = fp/(1 - fp) = 9$, muy cerca del 8 medido. En el ejemplo de 16 números de clase el óptimo fue p = 4 porque el trabajo era tan pequeño que el overhead del árbol pesaba enseguida. Aquí el trabajo paralelo es grande, así que conviene más hilos, pero el óptimo sigue estando por debajo de p_max: 7 núcleos más solo compran un 10 % de rapidez.

1.8 El jefe y el servidor de 64 núcleos. fp = 18/20 = 0.90.

p	Tp (s)	S	E	C (núcleo·s)	F (s^{-1})
1	20.00	1.00	1.00	20	0.050
9	4.00	5.00	0.56	36	0.139
16	3.13	6.40	0.40	50	0.128
64	2.28	8.77	0.14	146	0.060
∞	2.00	10.0	→ 0	∞	→ 0

Respuesta al jefe: “Con 64 núcleos el programa baja de 20 s a 2.3 s, pero en promedio 55 de los 64 núcleos estarían ociosos (E = 14 %), y la efectividad es menos de la mitad de la de 9 núcleos. Con 16 núcleos tenemos el 73 % de ese speedup a un tercio del costo. Ningún servidor baja de 2 s: el techo es 10×. Si necesitamos más velocidad, primero hay que reducir los 2 s de lectura (disco más rápido, leer en paralelo por bloques, solapar lectura con cómputo).” Nuestros datos van en la misma línea: con fp = 0.90 obtuvimos el 84 % de Amdahl, así que en la práctica los 8.77 de 64 núcleos serían menos.

4. Parte 2 — ¿Vale la pena paralelizar algo pequeño?

4.1 Predicción vs. medición

Pregunta	Predicción	Medido	Comentario
t_seq vs t_par(p=1)	diferencia < 2 %, secuencial algo más rápido	−0.41 % (OpenMP 1 hilo más rápido)	Dentro del ruido
S con N = 16, p = 15	0.01–0.05 (20–100× más lento)	≈ 1×10^{-5} (~100 000× más lento)	Subestimé el costo de una región paralela por 1000×
N donde conviene paralelizar	10 000 – 65 536	~17 000 (p = 2, 4); ~115 000–135 000 (p = 8, 15)	Acerté para pocos hilos; con muchos, el cruce se aleja
S con N = 16.7 M	4–6 con p = 15	7.10 (p = 8), 6.83 (p = 15)	Mejor de lo esperado, pero se estanca desde p = 8

4.2 2a — El Ts honesto

2.1 Con `amdahl_fp100.csv`, p = 1: t_seq = 0.302829 s, t_par = 0.301577 s. La versión OpenMP con 1 hilo fue 0.41 % más rápida, una diferencia dentro del ruido. Con los otros fp la diferencia va de +0.5 % (t_par más lento) a −2.3 %. El outlier de fp = 0.95 tiene un t_seq 12.8 % más alto, que es ruido. Con 0.3 s de trabajo, el costo de abrir una región paralela (µs) no se nota. En la suma sí se nota: con N = 16, la versión OpenMP con 1 hilo tarda 54 ns frente a ~1 ns de la secuencial (54× más lenta).

2.2 Sale inflado. La versión paralela con 1 hilo arrastra el overhead de OpenMP, así que, en general, es más lenta que la secuencial. Usarla como Ts agranda el numerador. En la suma con N = 16 inflaría S por 54×. Es

tramposo porque compara el programa contra una versión empeorada de sí mismo y no contra lo mejor que existe sin paralelismo, que es la alternativa real del usuario. Aquí, con fp = 1.00, la diferencia es mínima (S(15) = 7.14 en vez de 7.17), pero el método sigue estando mal.

4.3 2b — Sumar N números

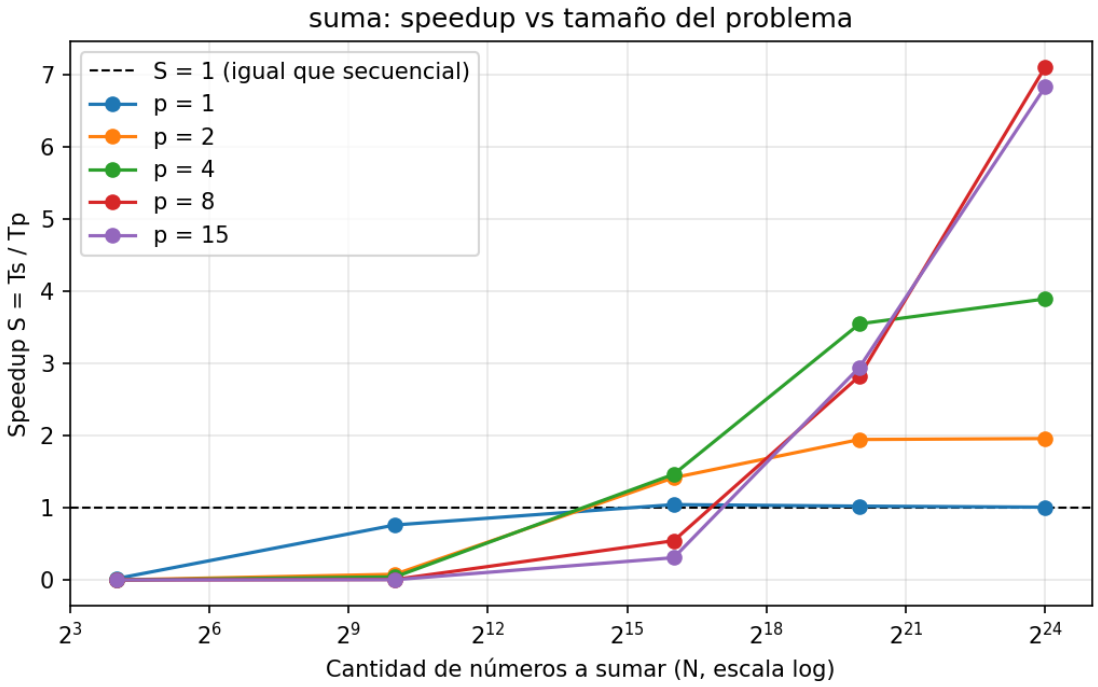


Figura 3: Speedup, modo suma

Speedup S = Ts/Tp (`suma_con_S_E.csv`):

N \ p	1	2	4	8	15
16	0.019	~0	~0	~0	~0
1 024	0.761	0.079	0.042	0.007	0.004
65 536	1.043	1.416	1.463	0.543	0.308
1 048 576	1.022	1.945	3.546	2.823	2.945
16 777 216	1.008	1.957	3.891	7.101	6.828

Tiempo de una suma paralela Tp (µs). Con N pequeño, esto es casi todo overhead:

N \ p	1	2	4	8	15
16	0.054	4.6	9.6	55.3	108.1
1 024	0.57	5.5	10.2	58.1	108.8
65 536	32.8	24.2	23.4	63.0	110.9

2.3 N = 16, p = 15: Ts ≈ 1 ns y Tp = 108 µs, así que S ≈ 1×10^{-5}. La versión paralela es ~100 000 veces más lenta. El CSV imprime Ts con 9 decimales, así que “1 ns” en realidad está entre 0.5 y 1.5 ns. El orden de magnitud no cambia.

2.4 El modelo de dos fases supone que solo cuestan las sumas: que lanzar los hilos, repartir, sincronizar y combinar es gratis, y que el árbol de log2 p sumas es todo el costo de combinar. En esta máquina una suma cuesta menos de 1 ns, y la región paralela cuesta 5 µs con p = 2 y 108 µs con p = 15, es decir, entre 10^4 y 10^5 sumas. Con N = 16 y p = 4 el modelo predice Tp = 5 sumas (~2 ns) y medimos 9.6 µs. Es la fuga η_c (comunicación y sincronización).

2.5 Lo que hace OpenMP en cada `parallel for` además de sumar: (1) despertar los p − 1 hilos del pool (o crearlos la primera vez), (2) que cada hilo calcule su rango de iteraciones, (3) inicializar la copia privada de `s`, (4) sumar, (5) combinar las p copias del `reduction`, (6) esperar a todos en la barrera implícita y (7) devolver los hilos al pool, donde esperan activos un rato y luego se duermen. También se mueven líneas de caché entre núcleos, y entre los dos clústeres si p > 5. Con N = 16, prácticamente todo el tiempo se va en (1), (5) y (6). Lo confirma que Tp sea casi igual con N = 16 y N = 1024 para cada p (4.6 frente a 5.5 µs con p = 2, y 108.1 frente a 108.8 µs con p = 15): el trabajo útil no se nota. El salto de 10 µs (p = 4) a 55 µs (p = 8) indica que despertar hilos que estaban dormidos, y en otro clúster, cuesta mucho más.

2.6 En este equipo, interpolando en log2 N, S cruza 1 cerca de N ≈ 17 000–18 000 con p = 2 o 4 y cerca de N ≈ 115 000–135 000 con p = 8 o 15. Regla práctica: un **`parallel for`** vale la pena si el trabajo de la región es al menos ~10 veces su costo fijo. Aquí eso significa unos 50–100 µs de trabajo para 2–4 hilos y más de 1 ms para 15. Además, el número de hilos debe crecer con el problema: con N = 1 M, p = 4 (S = 3.55) le gana a p = 15 (S = 2.95). Usar todos los núcleos no siempre es lo mejor.

2.7 Con N = 16.7 M no se llega a S = p: S = 7.10 con p = 8 y 6.83 con p = 15, así que agregar 7 hilos más empeora el resultado. El arreglo ocupa 128 MB y no cabe en ninguna caché, de modo que todo sale de la RAM. La versión secuencial lee a 15 GB/s, limitada por la cadena de sumas dependientes. La paralela sube a 30, 60 y 109 GB/s con p = 2, 4 y 8, y con p = 15 se queda en 104 GB/s. Ese techo de ~105 GB/s es el ancho de banda de memoria que alcanzan los núcleos de CPU, un recurso compartido por todos. Con muy poco cálculo por byte (1 suma por cada 8 bytes), unos 8 hilos ya lo saturan. No encaja en ninguna de las cuatro fugas: no es parte secuencial, ni sincronización, ni desbalance, ni trabajo redundante; es contención por un recurso de hardware. (De paso, con N = 1 M, que son 8 MB y sí caben en la L2 de 16 MB del clúster Super, p = 4 da 3.55 de los 4 posibles.)

5. Validez de las mediciones

- La laptop corrió con batería y con procesos activos (Terminal, Jupyter, WindowServer y un proceso del sistema al 99 % de CPU). La guía pide conectarla a la corriente y cerrar las apps. No se cumplió del todo, y eso afecta sobre todo los valores con p ≥ 9.
- Para medir la variación, se corrió fp = 1.00 tres veces más (`resultados/repeticiones/`). Hasta p = 8, S se repite con diferencias de ±0.1. Con p = 9–15 varía hasta ±3. Los valores oficiales de este reporte son los de la corrida del notebook.
- Ts con fp = 0.95 es un outlier (+12 %). Su S(15) está inflado en la misma proporción.

6. Parte 3 — Desbalance de carga

A cargo del integrante B (ver *`REPARTO.md`*): predicción, *`desb_static/dynamic/guided.csv`*, gráficas *`desbalance_*.png`* y preguntas 3.1–3.5.

7. Parte 4 — Problema que crece con los núcleos

A cargo del integrante B: predicción, *`gustafson.csv`*, gráficas *`gustafson_*.png`*, preguntas 4.1–4.5 y el segundo modelo (Gustafson).

8. Síntesis

Parte	Fuga principal	Número medido que lo demuestra
1 (fp < 1)	η_f parte secuencial	`t_parte_seq` no baja con p: 0.159 s con fp = 0.50 tanto en p = 1 como en p = 15. En p = 15 es el 90 % de Tp (fp = 0.50) y el 54 % (fp = 0.90). S(15) = 1.67 contra un techo de 2.
1 (fp = 1.00, p ≥ 9)	η_b desbalance (hilos en núcleos ocupados por otros procesos o más lentos, con `static`)	E = 1.01 con p = 8 → 0.48 con p = 15. fp efectivo = 0.922 en vez de 1.00.
2 (N pequeño)	η_c sincronización	Una región paralela vacía cuesta 4.6 µs (p = 2) y 108 µs (p = 15) frente a ~1 ns de trabajo: S(16, 15) ≈ 10^{-5}. Conviene paralelizar desde N ≈ 1.2×10^{5} con 15 hilos.
2a	— (ninguna)	t_par(1) − t_seq = −0.41 %: el overhead de la región no se ve con 0.3 s de trabajo.
3	(integrante B)	
4	(integrante B)	

η_r (trabajo redundante): no apareció en las Partes 1 y 2. Lo más parecido es combinar el `reduction`, pero son p sumas, despreciables.

Lo que no encaja en ninguna de las cuatro fugas:

1. Ancho de banda de memoria (2.7): la suma se estanca en ~105 GB/s y S(8) = 7.10 > S(15) = 6.83.
2. Frecuencia y potencia (1.1): un hilo solo corre más rápido que 15 juntos, y más aún con batería. Ts sale “demasiado bueno” y S baja sin que el programa tenga ninguna fuga.
3. Interferencia del sistema operativo (1.1): otros procesos ocupan núcleos. Su efecto es un desbalance, pero la causa está fuera del programa, y no se puede controlar desde el código porque macOS no deja fijar hilos a núcleos.
4. Ruido de medición: el outlier de Ts con fp = 0.95 dio S(2) = 2.13 > p, un “superlineal” falso.

9. Reto opcional

No se hizo.